Taxi Sharing in the City

A Multi-Agent Based Simulation in Mesa

Ludovica De Martino, Claudio Bendinelli
Université Paris-Saclay — Foundations of Agent-based Systems

1 The question

The city council wants to test shared taxis to reduce pollution, and asks what this changes for the traffic, the travel time of the passengers and the acceptability. Before writing any code we fixed what we want to observe, because the model depends on the question. We turned these three words into measures that the simulation can give:

- **Traffic / pollution:** the total km driven by the taxis and the km per passenger served. We also look at the occupancy: the average number of passengers in a moving taxi.
- **Travel time:** the wait (steps from the first call to the pickup) and the detour: km really driven with the passenger divided by the km of the direct trip (1.10 means a ride 10% longer).
- **Acceptability:** the % of passengers who give up because no taxi takes them in time, the % who arrive late, together with the wait and the detour. In our model a passenger also says how much it accepts (Section 3), so a taxi should never give a trip that its passenger would refuse.

We built the model step by step, with four design levels: (1) one passenger per taxi, (2) two passengers without changing the route, (3) two passengers with route change and several stops, (4) more seats (Table 1). We have no real taxi data, so the model cannot be validated against the real city: we use it to compare the levels with each other (Section 6).

2 The model

Environment. The city is a 20×20 grid; one cell is one district, and we count 1 km per district. Several agents can be in the same cell. Each cell has 8 neighbours, and a taxi crosses one district per step by moving to the neighbour that is the closest to its next stop: with 8 neighbours this is already a shortest path, so we need no routing algorithm. Getting in or out of the taxi takes one step. New passengers appear in random districts (0.3 per step for low demand, 0.8 for high demand), with a random destination at least 2 km away (about 9.7 km on average). The environment is *partially observable* (an agent knows its position and its messages), *stochastic* (passengers appear at random and the agents act in a random order, so the same action does not always have the same result: an accept can end with a failure if another passenger was faster; only a taxi that drives always crosses one district), *dynamic* (passengers appear by themselves) and *discrete*. The system is *open*: passengers enter and leave the city during the run.

Behaviour. Both agents run the loop perceive → decide → act and do one action per step. Perception only reads the messages and updates the beliefs; all the choices are made in the decision. When a passenger accepts its offer, the taxi adds the new stops to its route at the beginning of its decision: this changes its plan (its intentions), not the city, so it is not an action and does not use the step.

- The **passenger** is a *cognitive agent*: it has beliefs (the offers it received, its booking) and "if state then action" rules, but no explicit goal and no plan. Its actions are: call the taxis, accept an offer, reject the other offers, get in, leave, or wait. It takes the offer that brings it to its destination first, and it is free to ignore the other offers. If it has no booking after `max_wait` = 10 steps, it gives up and leaves.
- The **taxi** is a *rational agent*: its goal is explicit (bring each booked passenger to its destination within its limits) and it plans how to reach it: its route, the list of stops to serve, is its plan. To add a new passenger it tries every place for the pickup and every later place for the drop-off, without changing the order of the other stops, and keeps the shortest route that respects the seats and the limits of every passenger. Our routes have only a few stops, so trying all the places is fast. At each step the taxi chooses by priority: (1) if a stop is in its district, tell the passenger that it is here or that it has arrived; (2) tell the passengers it cannot take any more; (3) answer the callers it can take; (4) drive one district; (5) wait.

Level	Seats	A taxi that already has passengers takes a new one if…
1	1	never (today's taxi)
2	2	its route does not get longer (the new trip is "on the way")
3	2	every passenger keeps its limits: detour accepted and latest arrival
4	4	same as level 3, with 4 seats

Table 1: The four design levels.

Interaction. The interaction is direct: the agents exchange messages with the Contract Net Protocol (Figure 1). A passenger sends one `cfp` to all the taxis (the list of taxis is public in the city), with its trip, its limits and its address. Each taxi answers `propose` with the arrival time to every caller it can take before the deadline of the cfp (the next turn of the passenger); a taxi that cannot take the caller lets the deadline pass. The FIPA protocol allows this: a participant can send `refuse` or simply not answer before the deadline. We do not send `refuse` because a message costs a step (Section 3), and a taxi that refuses 10 callers would not drive. The passenger sends `accept-proposal` to the best offer and `reject-proposal` to the others. A proposal does not book the seat, so when the accept arrives the taxi checks again: if the passenger does not fit any more (another passenger took the place in between), it answers `failure` and the passenger calls again. At the pickup the taxi sends `inform` ("I am in your district") and the passenger decides to get in; at the drop-off it sends `inform-done` ("you have arrived", with the km of the ride). Both are assertive speech acts: the taxi only tells a fact.

Scheduling. Mesa runs the agents one after the other, in a new random order at each step. A message is read at the next turn of its receiver, so an answer arrives after one or two steps. A passenger decides with the offers it has at its next turn: if it has none, it calls again. An answer that arrives later is not lost: it is kept with the answers to the new call, because it is still a real offer for the same trip.

3 Our main choices and why

1. **A message is an action.** In Mesa all the agents run in turns, and sending a message means asking the model to write in the mailbox of the receivers: it changes the environment during my turn, like driving. So in one step an agent sends one message or does one physical action (drive, get in…), never both. One message can have several receivers: the cfp goes to all the taxis, and one propose answers all the callers (it contains one arrival time for each of them, by address). Without this, a taxi could answer only one caller per step and

many passengers would give up only because of this rule. The price is real: a taxi that proposes does not drive in that step.

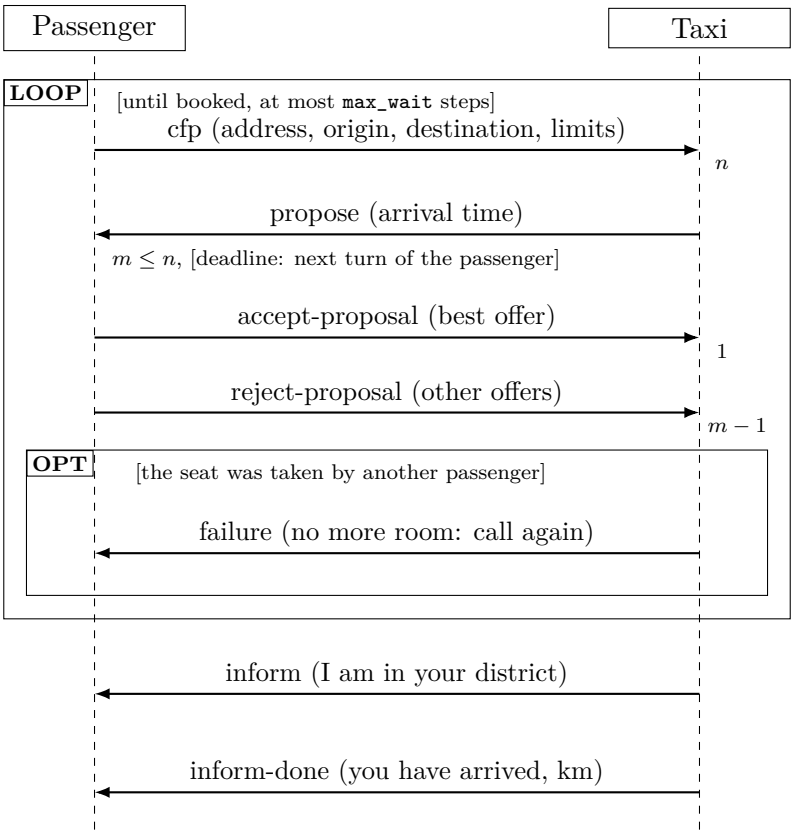


Figure 1: The Contract Net Protocol between a passenger and the n taxis (AUML). Each arrow is one message, so one action of its sender. A taxi that cannot take the passenger lets the deadline pass, so $m \leq n$ taxis propose.

2. **The passenger says what it accepts.** In its cfp the passenger sends two limits: the longest ride it accepts (direct trip + `max_detour` = 3 km) and the latest step at which it wants to arrive (`max_wait` steps for the pickup + its longest ride + 2 steps to get in and out). The taxi checks the limits of all its passengers before adding a new one. Without the second limit a taxi could delay a booked passenger again and again to take new ones. We chose this because acceptability is the passenger's choice, not the taxi's.

3. **A proposal does not book the seat.** This keeps the taxi simple (it does not need to remember its offers). But a free taxi answers all the callers in one propose, several of them accept, and only the first one fits: the others get a failure. We measured it (Section 5.1): at high demand this happens often, so it is not a detail.

4. **Same passengers for every setting.** All the passengers of a run are drawn at the start, before the taxis. So with the same seed every level and every fleet gets exactly the same passengers (same average trip, 9.63 km at high demand, for every level), and we can compare the settings run by run.

5. **A margin in the plan of the taxi.** Because of choice 1, a taxi loses a step each time it sends a message, and it cannot know in advance how many calls it will answer. So we added a parameter `margin`: the extra steps a taxi counts for each stop when it plans. With `margin` = 0 (our default) the taxi plans 1 step per km and 1 step per stop. Section 5.5 shows what this changes.

4 Implementation in Mesa

The code has three files in `taxi_share/` (`messages.py`, `agents.py`, `model.py`), plus `app.py` (live view), `experiments.py` and `checks.py`.

- **Encapsulation.** Python does not stop an agent from reading another one, so we were careful: an agent never reads or changes the variables of another agent. It only knows the others by their address (the sender of a message, or the address that a passenger writes in its cfp, like an AID in JADE), sends messages and uses the services of the model (deliver a message, leave the city). Only the data collector and the live view read the agents, and they take no decision.
- **Procedural loop.** `step()` only calls `perceive()`, `decide()` and `act()`. `perceive()` only reads the messages. `decide()` returns one action; the taxi also updates its plan there with the accepted offers, but `decide()` never changes the city. `act()` does the action, with one branch per action.
- **Synchronous runtime.** The longest computation is the plan of a taxi, which is short, so no agent slows down the others.
- **Direct communication.** Mesa has no messages, so we wrote a `Message` class (performative, content) and a `MessageReceiver` (a `CellAgent` with a list of messages and a `notify(message, sender)` method). The model delivers each message by calling `notify` on every receiver. Messages contain only data (coordinates, numbers, addresses).

The live view (`solara run app.py`) shows the grid, sliders for the main parameters (also the margin) and three charts (passengers, km, and % who give up / share / arrive late). Figure 2 shows the grid.

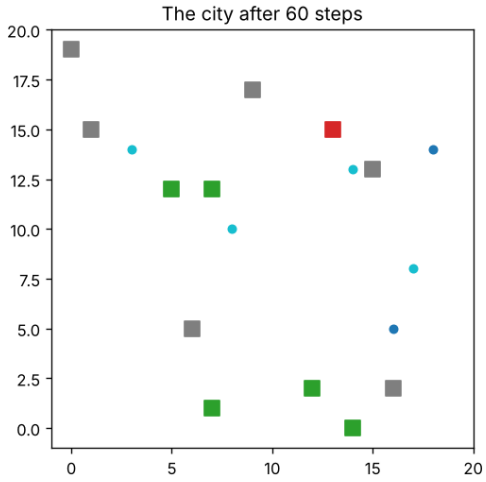


Figure 2: The city after 60 steps (level 4, high demand). Squares are taxis (grey: empty, green: one passenger, red: two or more), dots are passengers (blue: calling, cyan: booked).

5 Results

Every setting is run 10 times (seeds 0 to 9). Passengers appear during 300 steps, then the run goes on until every passenger has left. We give the mean of the 10 runs and, in brackets or as bars in the charts, the standard deviation. Since two settings with the same seed have the same passengers, we also count in how many of the 10 runs a setting is better than another one.

Unless written otherwise there are 12 taxis, `max_wait` = 10 and `margin` = 0. All the numbers are in `results/tables.txt`.

How to read the measures. (1) The wait is measured only for the passengers who are served: the others give up, so with `max_wait` = 10 the wait cannot grow much above 10 steps. When taxis are scarce, the % who give up is the real measure of the service. (2) With 12 taxis, a demand of 0.8 passengers per step is more than the fleet can serve: even with sharing about one passenger out of five gives up. The high-demand results describe a city where taxis are scarce.

5.1 The four levels

	Low demand (0.3 / step)				High demand (0.8 / step)			
Level	1	2	3	4	1	2	3	4
Passengers served	88.3	88.5	88.7	88.6	179.6	182.1	188.7	187.7
% gave up	2.2	2.0	1.8	1.9	25.7	24.6	21.9	22.3
Total km	1267	1270	1210	1212	2730	2729	2545	2536
km per passenger	14.35	14.35	13.64	13.67	15.21	14.99	13.49	13.52
Occupancy	0.68	0.68	0.72	0.72	0.62	0.64	0.73	0.72
% shared	0.0	0.9	11.7	11.0	0.0	3.8	29.2	27.4
Wait (steps)	6.9	7.0	6.8	6.9	9.7	9.7	9.7	9.8
Detour	1.00	1.00	1.01	1.01	1.00	1.00	1.03	1.03
% arrived late	0.0	0.0	1.7	1.6	3.6	3.4	21.6	22.0
% accepts that failed	5.1	4.9	4.6	4.8	41.0	40.5	30.8	31.9

Table 2: The four levels (12 taxis, margin 0, mean of 10 runs). Standard deviations: about 1.4–2.7 for the % who give up, 0.3–0.6 for the km per passenger.

Table 2 and Figure 3 give the results.

- **Level 3 is the real step.** At high demand, from level 1 to level 3 the km per passenger fall by 11% (15.21 → 13.49), the total km by 7%, and the passengers who give up from 25.7% to 21.9%, so 9 more passengers served with fewer km. Level 3 is better than level 2 in 10 runs out of 10 on both measures, so this effect is solid.
- **Level 2 changes almost nothing.** Two trips exactly "on the way" are rare (3.8% of rides shared). Level 2 is better than level 1 in only 6 or 7 runs out of 10: the difference is smaller than the noise, so we do not trust it.
- **More seats change nothing (level 4 ≈ level 3).** On the % who give up, level 4 is better in 2 runs, worse in 4 and equal in 4: the small difference (22.3% against 21.9%) is noise. Section 5.3 shows why: a fourth passenger never fits.
- **Many accepts end with a failure.** At high demand 41% of the accepted offers end with a failure at level 1, and 31% at level 3 (only about 5% at low demand). A free taxi proposes to all the callers at the same time, several accept, and only the first one gets the seat: the others must call again and lose time. A level-3 taxi suffers less, because it has a second seat for the second caller. So part of the gain of sharing in the % who give up comes from our protocol, and a real taxi app that books the seat would make level 1 a bit better.
- **The detour is small, but many passengers arrive late.** The detour goes from 1.00 to 1.03 (about 0.3 km on a 9.6 km trip) and the wait of the served passengers does not change. However 22% of the passengers of levels 3 and 4 arrive after their latest step (against 3.6% at level 1), even if the taxi checked this limit. The reason is our choice 1:

the taxi plans with 1 step per km and per stop, but it also spends steps sending messages, and a shared taxi sends many more of them (every pickup, drop-off and proposal is one step). So its promise is optimistic. We study this in Section 5.5.

- **Low demand.** There are enough taxis, sharing is rarer (11% of the rides) and the km per passenger fall by 5% only. About 2% of the passengers still give up; we did not study in detail which ones.

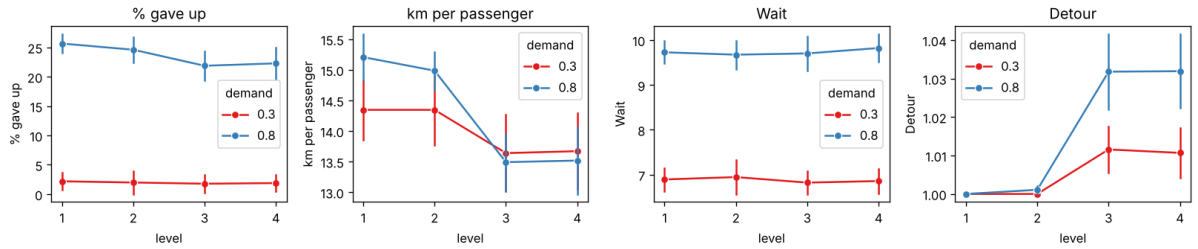


Figure 3: Main measures per level, for low and high demand (12 taxis, mean and standard deviation of 10 runs).

5.2 Size of the fleet

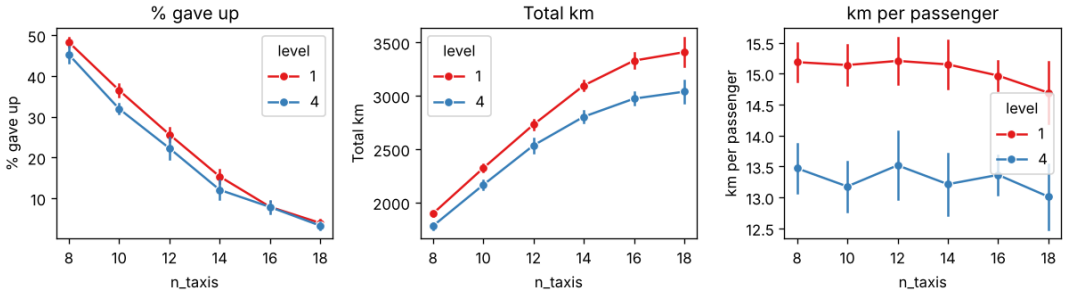


Figure 4: Levels 1 and 4 with 8 to 18 taxis (high demand, mean and standard deviation of 10 runs).

Figure 4 shows that the total km depend first on the number of taxis: a taxi that has work drives. Two cases are interesting (Table 3):

- **With many taxis (16)**, both levels lose the same 7.9% of the passengers, and sharing serves the same number of passengers with 11% fewer km (3330 → 2974). This is the cleanest result for pollution: the same service for less driving.
- **With scarce taxis (8 to 14)**, sharing serves 7 to 11 more passengers and still drives 6–9% fewer km.

	Passengers served	% gave up	Total km	km per passenger
Level 1, 16 taxis	222.5	7.9	3330	14.97
Level 4, 16 taxis	222.6	7.9	2974	13.36
Level 1, 12 taxis	179.6	25.7	2730	15.21
Level 4, 12 taxis	187.7	22.3	2536	13.52

Table 3: Same fleet, with and without sharing (high demand, mean of 10 runs).

The km per passenger stay around 15 at level 1 and around 13.0–13.5 at level 4 for every fleet size. But sharing does not replace taxis: up to 16 taxis, two more taxis serve 18 to 28 more passengers at level 1 and 10 to 32 more at level 4, while sharing adds at most 11. So sharing is not a way to cut the fleet a lot; it is a way to serve the same people with fewer km.

5.3 Detour accepted and seats

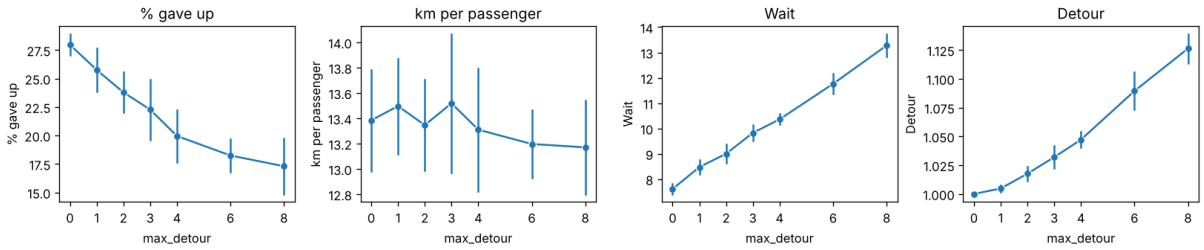


Figure 5: Level 4 when passengers accept a detour of 0 to 8 km (high demand, 12 taxis, 10 runs).

The detour that passengers accept is the main parameter of sharing (Figure 5). From 0 to 8 km, the rides shared go from 6% to 54% and the passengers who give up from 28.0% to 17.3%. But the passengers pay: the wait goes from 7.6 to 13.3 steps and the detour from 1.00 to 1.13. The km per passenger almost do not move (13.38 → 13.17): a larger detour lets the taxis serve more people, but each shared ride is longer. So the detour is a choice between service (fewer people without a taxi) and comfort (shorter trips), and the council has to make it. Note that with 0 km the latest arrival is also stricter, which is why more passengers give up than at level 2.

Seats. We tried 2, 3, 4 and 6 seats with a detour of 3 and of 8 km (Figure 6). Nothing changes: with 3 km the % who give up stays between 21.9 and 22.3 and the occupancy at 0.72–0.73; with 8 km it stays between 17.3 and 18.1 and the occupancy at 0.80–0.81. With 3 km, 3, 4 and 6 seats give exactly the same numbers in all the runs: a fourth passenger never fits. With 8 km, 4 and 6 seats give the same numbers: a fifth passenger never fits. Because every passenger keeps its own limits, two seats are almost always enough. This is a consequence of our rule: passengers who accept long trips would make more seats useful.

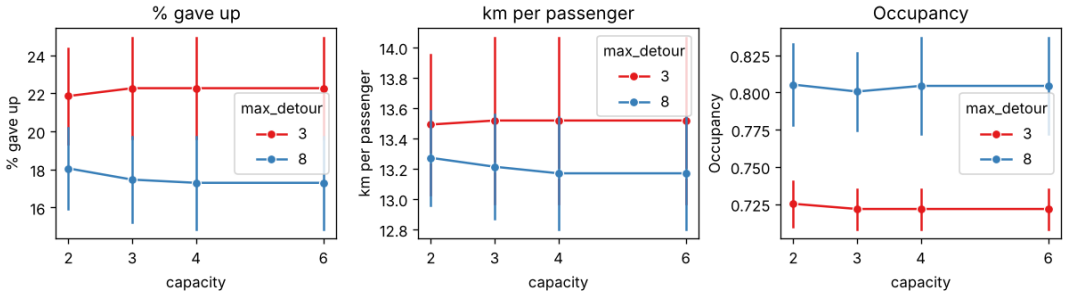


Figure 6: Level 4 with 2 to 6 seats, for a detour of 3 and 8 km (high demand, 12 taxis, 10 runs).

5.4 Patience of the passengers

The value `max_wait` = 10 is our choice, not a measure, so we tested 5 to 30 steps (Figure 7). The % who give up depends a lot on it, so its absolute value must not be trusted. And the gain of sharing on the km depends on it too:

- **At level 1, more patience is worse after 10 steps** (25.7% → 29.0% give up at 20 steps) and the km per passenger grow from 13.2 (5 steps) to 15.2 (10 steps) and 17.4 (20 steps). A patient passenger can be picked up by a far taxi: the taxi drives many empty km to reach it and serves fewer people.
- **At level 4, more patience serves more people** (22.3% → 12.9% give up at 30 steps) and the km per passenger stay at 12.7–13.6: a taxi that already drives can pick up a patient passenger on its way. Level 4 is better than level 1 in 9 or 10 runs out of 10 for every value from 10 to 30.
- **But the served passengers wait longer at level 4.** With 20 and 30 steps of patience they wait 18.3 and 26.1 steps, against 15.5 and 19.5 at level 1: a shared taxi often picks up a patient passenger only after its other stops. For acceptability, sharing serves more people, but later.

So the gain of sharing on the km per passenger is not one number: it is 3.6% with 5 steps (13.16 → 12.69, still better in 9 runs out of 10), 11% with 10 steps and 23% with 20 or 30 steps. Most of the difference comes from the empty km of level 1, which grow with the patience. With very impatient passengers (5 steps) the two levels also lose the same number of passengers (level 4 better in 5 runs out of 10): there is no time to find a shared ride.

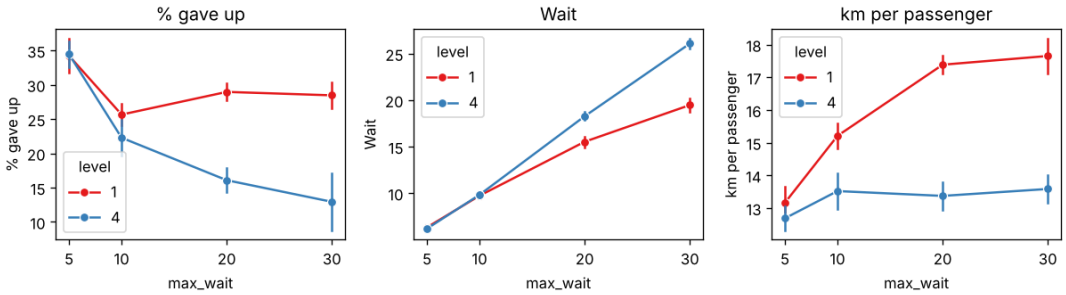


Figure 7: Levels 1 and 4 with `max_wait` from 5 to 30 steps (high demand, 12 taxis, 10 runs).

5.5 Can a taxi keep its promises? The margin

Section 5.1 showed that at levels 3 and 4 about 22% of the passengers arrive late, because the taxi forgets the steps it uses to send messages. We ran levels 1, 3 and 4 again with a margin of 0, 1 and 2 extra steps per stop (Table 4, Figure 8).

	Level 1			Level 3		
Margin (steps per stop)	0	1	2	0	1	2
% arrived late	3.6	0.0	0.0	21.6	1.1	0.0
% gave up	25.7	26.9	30.4	21.9	25.7	30.6
Passengers served	179.6	176.5	168.2	188.7	179.4	167.7
km per passenger	15.21	14.45	13.68	13.49	13.47	13.21
% shared	0.0	0.0	0.0	29.2	15.4	8.5
Wait (steps)	9.7	8.4	7.0	9.7	8.0	6.7

Table 4: Levels 1 and 3 with a margin of 0, 1 and 2 steps per stop (high demand, 12 taxis, mean of 10 runs). Level 4 gives almost the same numbers as level 3.

- **One step of margin is enough to keep the promises.** With margin 1 the late passengers fall from 21.6% to 1.1% at level 3 and from 3.6% to 0 at level 1 (better in 10

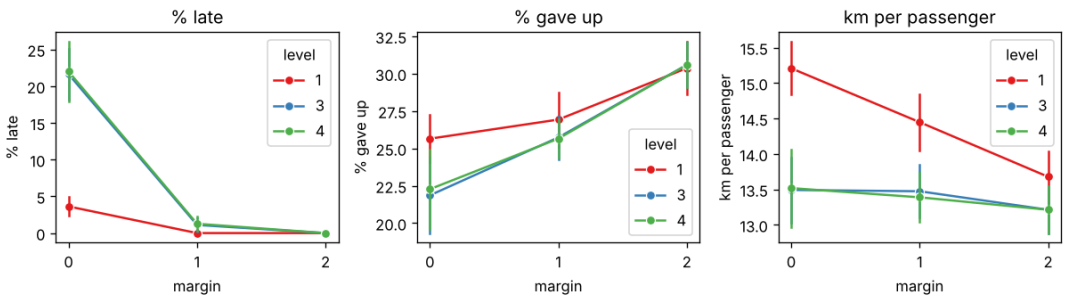


Figure 8: Levels 1, 3 and 4 with a margin of 0 to 2 steps per stop (high demand, 12 taxis, 10 runs).

runs out of 10 for every level). A taxi can only lose time when it sends a message instead of driving, so the 1% left are rides in which the taxi sent more messages than its margin covers.

- **But a careful taxi takes fewer passengers.** With a margin the taxi says "no" more often, so more passengers give up (21.9% → 25.7% at level 3) and fewer rides are shared (29% → 15%). The wait of the served passengers is shorter because a taxi with a margin does not propose to the passengers it would reach too late.
- **Sharing still wins, but less.** With margin 1, level 3 needs 7% fewer km per passenger than level 1 (14.45 → 13.47, better in 10 runs out of 10) and fewer passengers give up (26.9% → 25.7%, better in 8 runs out of 10). With margin 0 the gain was 11%: the drop comes from level 1 (15.21 to 14.45); level 3 does not change. So the fair comparison between levels is at the same margin: with `max_wait` = 10 the gain of sharing is between 7% and 11%.
- **Too much margin kills sharing.** With margin 2 nobody is late, but all the levels lose about 30% of the passengers and only 8% of the rides are shared: level 3 and level 1 serve the same people.

So a message that costs a step has a real price for the service, and a taxi company should keep a small margin (1 step per stop) if it wants its passengers to trust the arrival time.

6 Limits

- **Validity.** We have no real data to compare with, so the model is not validated: the numbers are not predictions, and we trust the direction of the effects more than their size. We checked that the code follows the model: `checks.py` verifies at every step that every agent does at most one action, that a taxi never has more passengers than seats and that no ride is longer than the passenger accepts, and that every passenger leaves the city. Only the latest arrival can be missed, because it is a plan of the taxi (Section 5.5).
- **Geometry.** With 8 neighbours a diagonal move costs 1 km, like a straight one. Real streets are longer, so all the km are a bit underestimated, but the comparison between levels is not changed.
- **Uniform demand.** In a real city there is a centre and rush hours, with many trips in the same direction. This would make sharing easier, so our gains are probably low.
- **No traffic jams, no private cars:** a taxi always crosses a district in one step, so we measure the traffic only with the km.

- **A message costs a step.** Taxis that propose do not drive in that step, and every passenger without an offer calls all the taxis again. This makes the communication a real cost (Section 5.5), but in a real city a central app would do part of this work.
- **A proposal does not book the seat.** This causes many failures at high demand (Section 5.1) and penalises level 1 more than level 3.
- **Simple passengers.** They do not walk to a nearby district to meet a taxi, all have the same limits, and they never change taxi during a trip. They count as on board as soon as the taxi says it is here, even if they get in one step later.

7 Conclusion

With the same fleet, shared taxis carry more passengers for each km: with our default patience (10 steps) 7% to 11% fewer km per passenger at high demand (7% when the taxis keep a margin so that almost nobody is late) and fewer passengers left without a taxi, for a small price: a ride 3% longer on average. With many taxis, sharing gives the same service with 11% fewer km, which is the gain the council is looking for. The size of this gain depends on the patience of the passengers: from about 4% when they wait 5 steps to about 23% when they wait 20 steps or more, but then the served passengers also wait longer than with today's taxis. The main decision is the detour accepted by the passengers: a larger detour serves more people but makes the trips longer. A second decision is how careful the taxis are: a small margin keeps the arrival time reliable, but each taxi accepts fewer people. Finally, with reasonable limits for the passengers, two seats per taxi are enough.